

Filter* yang Dapat Ditukar Saat Program Berjalan: Menerapkan Pola *Microkernel* pada Arsitektur *Pipe-and-Filter

Rancangan Penelitian, Arsitektur Perangkat Lunak (IF231303)

Abstrak

Pipe-and-filter banyak dipakai untuk membangun sistem pengolah data. Aplikasi pengolah gambar, proses ETL (Extract, Transform, Load), dan *compiler* semuanya berbentuk *pipeline*. Data masuk di satu ujung, lalu melewati beberapa tahap, kemudian keluar di ujung lain. Setiap tahap disebut *filter*. *Filter* mudah ditulis dan mudah diuji satu per satu, tetapi susunannya sulit diubah. Pada sebagian besar program, urutan *filter* sudah ditetapkan saat program dikompilasi. Menambah atau menukar satu tahap berarti membangun ulang seluruh *pipeline*. Pola *microkernel* menyelesaikan masalah serupa dengan cara lain. Inti program dijaga tetap kecil, lalu fitur dimuat sebagai *plugin* saat program berjalan. Penelitian ini menggabungkan kedua pola tersebut, lalu mengukur waktu penggantian *filter* dan tambahan waktu yang timbul.

1. Pendahuluan

Pipeline adalah salah satu bentuk arsitektur paling tua dan paling berguna. Data mengalir satu arah melalui beberapa *filter*. Setiap *filter* mengerjakan satu tugas saja, misalnya membaca berkas atau mengubah ukuran gambar. Setiap *filter* hanya bergantung pada data yang diterimanya. Karena itu *filter* mudah ditulis, mudah diuji, dan mudah dipakai ulang.

Masalah muncul ketika *pipeline* harus diubah. Pada program biasa, *filter* dirangkai langsung di dalam kode. Urutannya sudah tetap sejak program dikompilasi. Jika ada kebutuhan menyisipkan tahap baru, maka kode harus disunting dan program harus dibangun ulang. Untuk *pipeline* yang berjalan terus-menerus, hal itu berarti program harus berhenti hanya demi perubahan kecil.

Pola *microkernel* sudah menangani masalah sejenis. Inti program hanya berisi bagian paling dasar. Setiap fitur tambahan berupa *plugin* yang dicari dan dimuat saat program berjalan. Editor teks dan peramban dibangun dengan cara ini. Penelitian ini bertanya apa yang terjadi jika *filter* diperlakukan sebagai *plugin*. Gagasan ini terdengar bagus, tetapi ada harganya. Memanggil *filter* lewat antarmuka *plugin* menambah satu lapisan perantara. Lapisan tersebut berada pada bagian kode yang mungkin dijalankan jutaan kali.

2. Pertanyaan Penelitian

Pertanyaan 1. Apakah tahap pada *pipeline* dapat ditambah, dihapus, dan ditukar urutannya saat program berjalan tanpa memasang ulang *pipeline*?

Pertanyaan 2. Berapa besar tambahan waktu yang timbul akibat pemanggilan *filter* lewat *plugin*?

3. Kontribusi

Penelitian ini menghasilkan rancangan *pipeline* yang susunannya dapat diubah saat program berjalan, dengan menggabungkan dua pola yang sudah dikenal luas. Gabungan kedua pola tersebut biasanya hanya dibahas sebagai gagasan. Angka biaya pemanggilan lewat *plugin* di dalam rangkaian *filter* juga belum tersedia. Hasil penelitian mengisi keduanya.

4. Tujuan

Pertanyaan pertama menyangkut kemungkinan menukar tahap *pipeline* saat program berjalan. Pertanyaan kedua menyangkut harganya. Penelitian ini tidak bermaksud membuktikan satu rancangan selalu lebih baik. Angka yang dihasilkan dimaksudkan agar pengembang dapat memutuskan sendiri untuk *pipeline* miliknya.

5. Metode

5.1 Teknologi yang Digunakan

Seluruh program ditulis dengan Python 3.12. Pengolahan gambar memakai Pillow. Pencarian dan pemuatan *plugin* memakai modul `importlib` serta *entry point* dari `importlib.metadata`. Kedua modul tersebut sudah tersedia di pustaka standar Python. Waktu diukur dengan `time.perf_counter` dan `pytest-benchmark`. Pemakaian memori diambil lewat `psutil`, sedangkan `watchdog` dipakai untuk mendeteksi berkas *plugin* baru. Versi seluruh pustaka dikunci dengan `uv`, agar percobaan dapat diulang dengan hasil yang sama.

- Python 3.12: bahasa yang dipakai untuk kedua versi program
- Pillow: pustaka pengolah gambar
- `importlib` dan `importlib.metadata`: modul bawaan Python untuk memuat *plugin* saat program berjalan
- `pytest-benchmark`: alat pengukur waktu eksekusi secara berulang
- `psutil`: pustaka pembaca pemakaian memori
- `watchdog`: pustaka pemantau berkas baru di dalam direktori *plugin*
- `uv`: alat pengunci versi pustaka agar percobaan dapat diulang

5.2 Langkah Penelitian

1. Buat *pipeline* pengolah gambar sederhana dengan minimal empat *filter*, misalnya baca berkas, ubah ke hitam putih, ubah ukuran, dan tambah *watermark*.
2. Buat versi pertama yang merangkai *filter* secara statis di dalam kode. Versi ini dipakai sebagai pembanding.
3. Buat versi kedua yang memuat setiap *filter* sebagai *plugin* lewat *entry point* `importlib.metadata`.
4. Pastikan kedua versi menghasilkan berkas keluaran yang sama persis untuk masukan yang sama.
5. Ukur waktu penggantian pada versi *plugin*. Tambah satu *filter*, hapus satu *filter*, lalu tukar urutan dua *filter* saat program sedang berjalan.
6. Ukur tambahan waktu dengan menjalankan kedua versi pada kumpulan gambar yang sama memakai `pytest-benchmark`.
7. Ulangi pengukuran cukup banyak kali, agar dapat dilaporkan median beserta rentangnya.
8. Bandingkan selisih kecepatan kedua versi, lalu sajikan berdampingan dengan waktu penggantian *filter*.

6. Metrik Evaluasi

- Waktu mengganti satu tahap tanpa memasang ulang program, dalam detik

- Tambahan waktu pemanggilan per *filter*, dalam mikrodetik
- Selisih kecepatan antara versi *plugin* dan versi statis, dalam persen
- Waktu memuat *plugin* saat program dimulai, dalam milidetik
- Jumlah baris kode yang berubah untuk menambah satu *filter* baru pada masing-masing versi